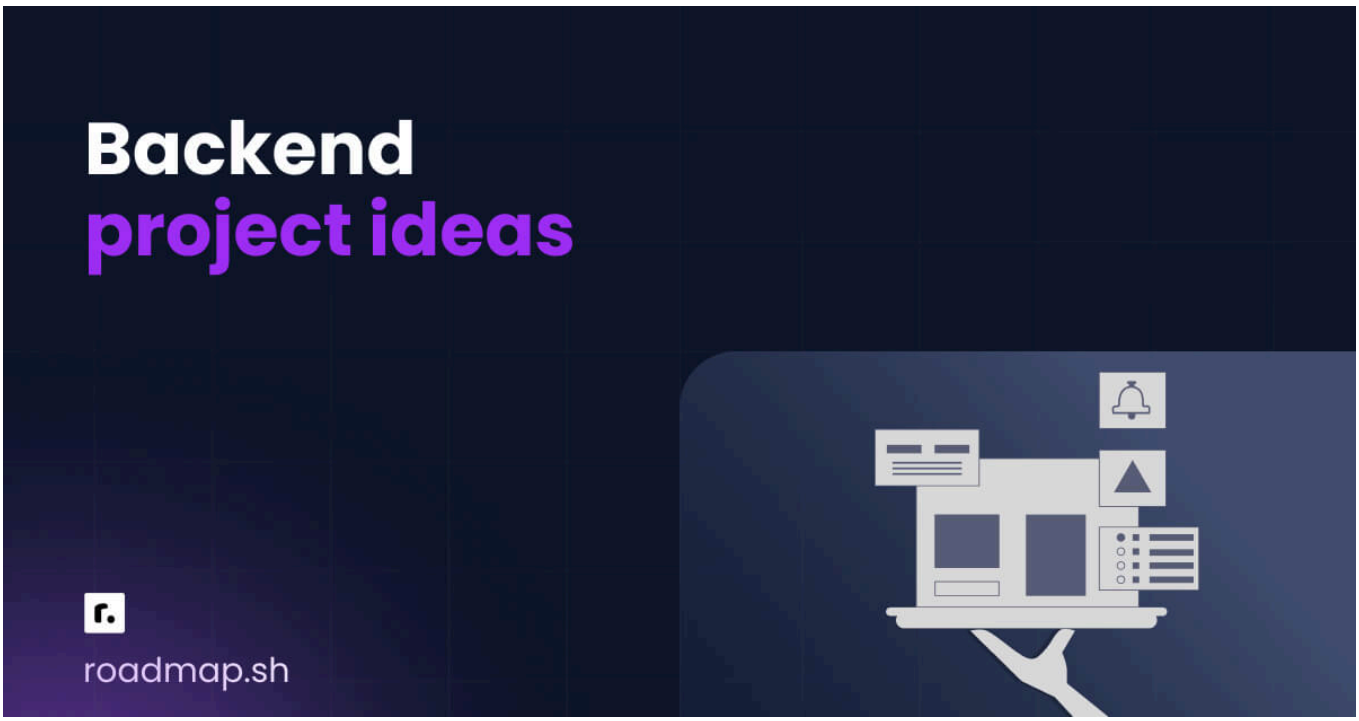
20 Backend Project Ideas to take you from Beginner to Pro

Fernando Doglio · Improve this Guide



As backend developers, showcasing our work to others is not straightforward, given that what we do is not very visible.

That said, having a project portfolio, even as backend developers, it's very important, as it can lead to new job opportunities.

As an added bonus, the experience you get out of the entire process of building the apps for your portfolio will help you improve your coding skills.

Let's take a look at 20 of the best backend projects you can work on to improve both your project portfolio and to **learn backend development**.

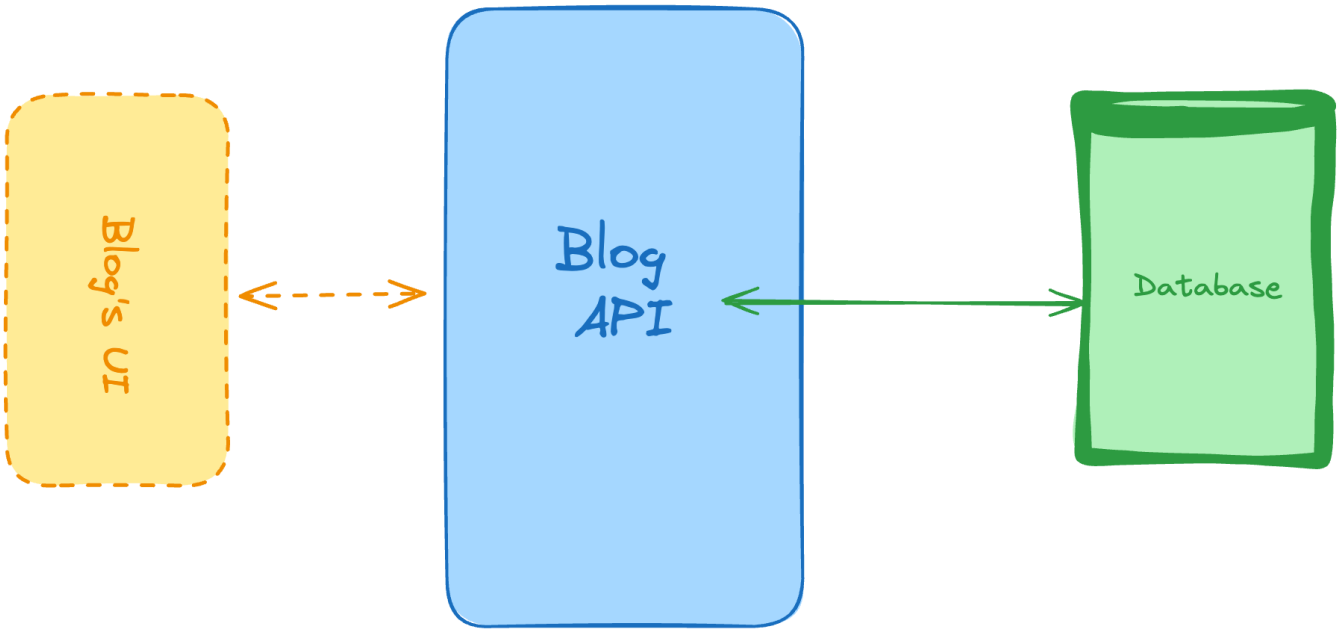
Keep in mind that these project ideas are organized from easiest to hardest to complete, and the entire list should take you at least a year to complete, if you're not rushing the process.

So sit back, grab a cup of your favorite hot drink, and let's get started!

1. Personal Blogging Platform API

Difficulty: Easy

Skills and technologies used: CRUD for main operations, databases (SQL or NoSQL), server-side RESTful API.



Let's start with a very common one when it comes to backend projects.

This is a RESTful API that would power a personal blog. This implies that you'd have to create a backend API with the following responsibilities:

- Return a list of articles. You can add filters such as publishing date, or tags.
- Return a single article, specified by the ID of the article.
- Create a new article to be published.
- Delete a single article, specified by the ID.
- Update a single article, again, you'd specify the article using its ID.

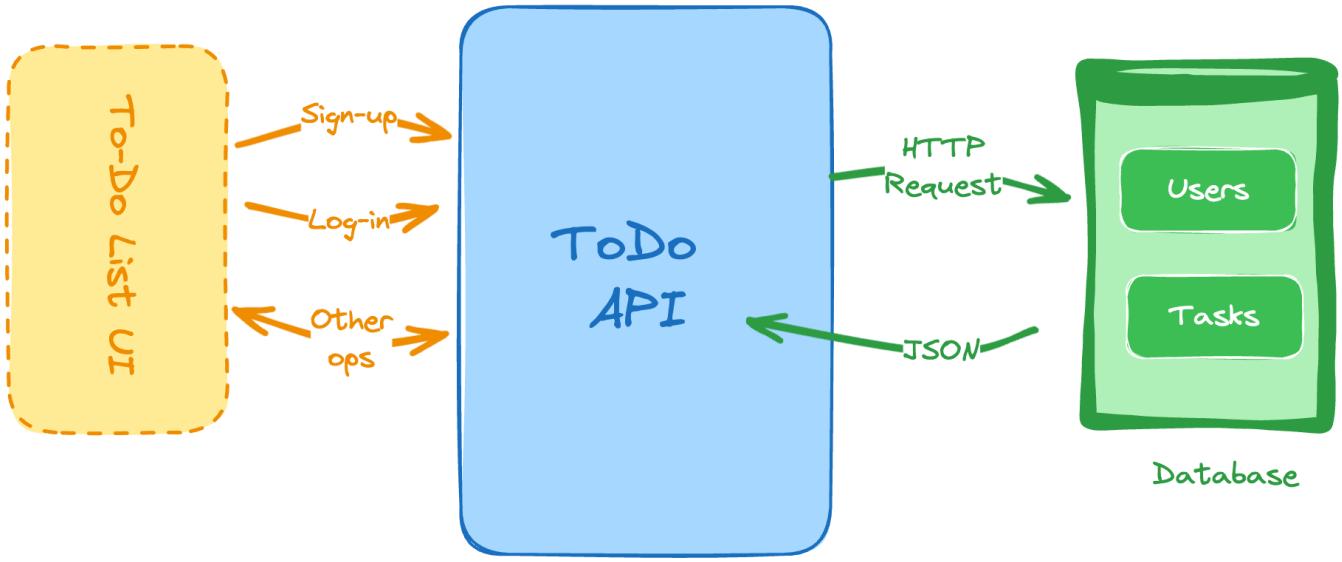
And with those endpoints you've covered the basic CRUD operations (**C**reate, **R**ead, **U**pdate and **D**elete).

As a recommendation for techstack, you could use **Fastify** as the main **backend framework** if you're going with Node, or perhaps **Django** for Python or even **Ruby on Rails** or **Sinatra** for Ruby. As for your database, you could use **MongoDB** if you want to try NoSQL or **MySQL** if you're looking to get started with relational databases first.

2. To-Do List API

Difficulty: Easy

Skills and technologies used: REST API design, JSON, basic authentication middleware.



We're continuing with the APIs for our backend project ideas, this time around for a To-Do application. Why is it different from the previous one?

While the previous project only focused on the main CRUD operations, here we'll add some more interesting responsibilities, such as:

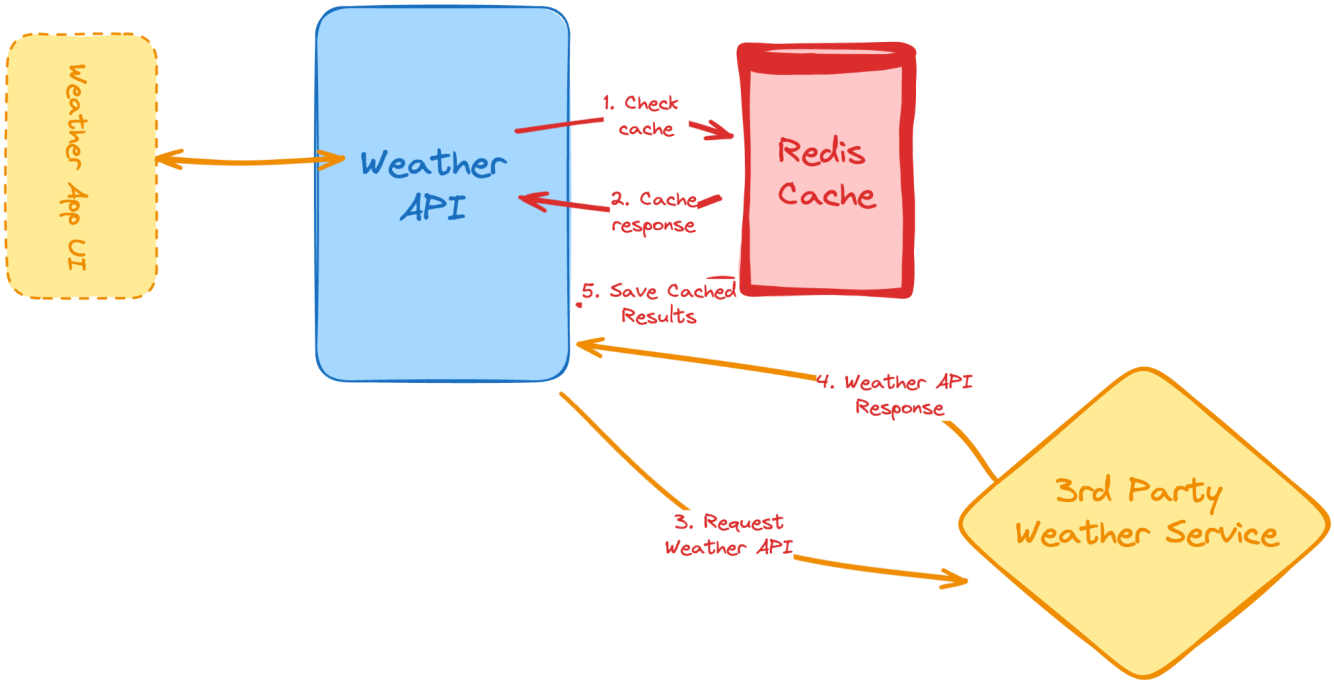
1. An authentication logic, which means you'll have to keep a new table of users and their credentials
2. You'll have to create both users and tasks.
3. You'll also have to be able to update tasks (their status) and even delete them.
4. Get a list of tasks, filter them by status and get the details of each one.

You're free to implement this with whatever programming language and framework you want, however, you could continue using the same stack from the previous project.

3. Weather API Wrapper Service

Difficulty: Easy

Skills and technologies used: Third-party API integration, caching strategy, environment variable management.



Let's take our API magic to the next level with this new backend project. Now instead of just relying on a database, we're going to tackle two new topics:

- Using external services.
- Adding caching through the use of a quick in-memory storage.

As for the actual weather API to use, you can use your favorite one, as a suggestion, here is a link to **Visual Crossing's API**, it's completely FREE and easy to use.

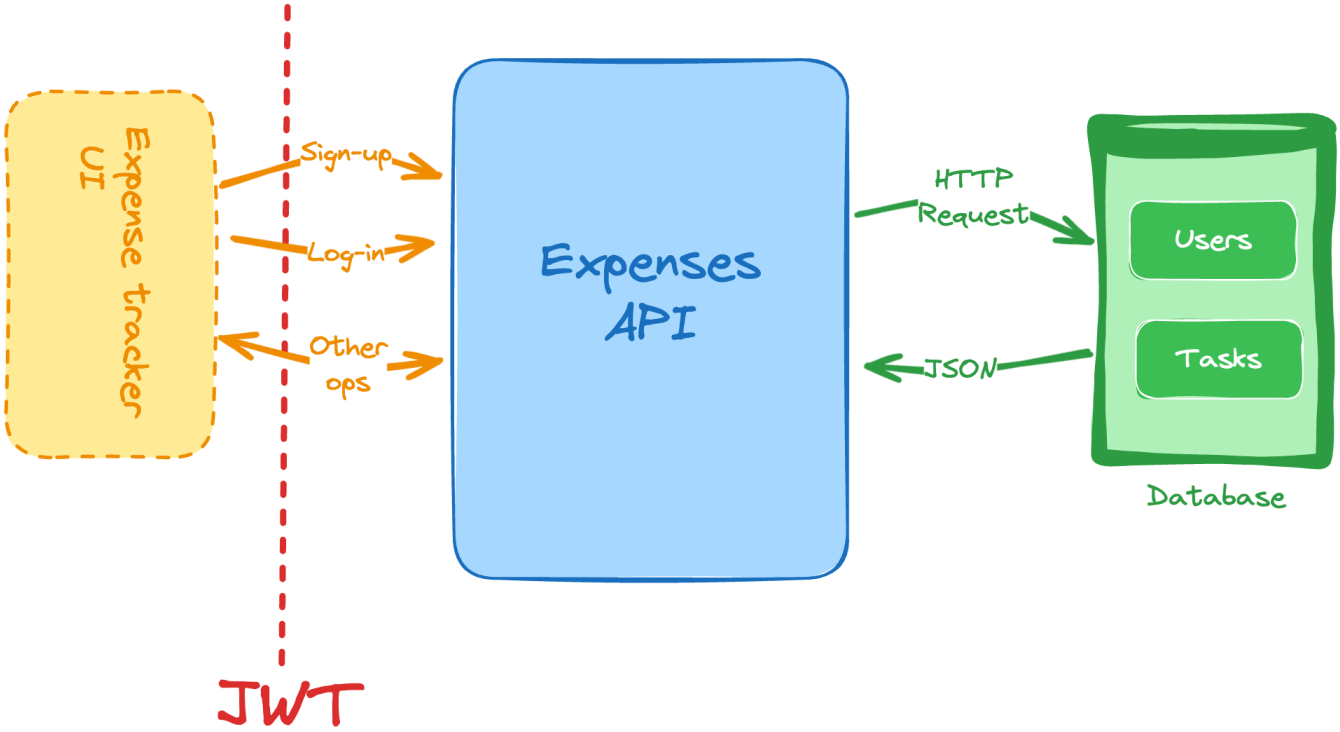
Regarding the in-memory cache, a pretty common recommendation is to use **Redis**, you can read more about it **here**, and as a recommendation, you could use the city code entered by the user as the key, and save there the result from calling the API.

At the same time, when you "set" the value in the cache, you can also give it an expiration time in seconds (using the EX flag on the **SET command**). That way the cache (the keys) will automatically clean itself when the data is old enough (for example, giving it a 12-hours expiration time).

4. Expense Tracker API

Difficulty: Easy

Skills and technologies used: Data modeling, user authentication (JWT).



For the last of our “easy” backend projects, let’s cover one more API, an expense tracker API. This API should let you:

- Sign up as a new user.
- Generate and validate JWTs for handling authentication and user session.
- List and filter your past expenses. You can add the following filters:
  - Past week.
  - Last month.
  - Last 3 months.
  - Custom (to specify a start and end date of your choosing).
- Add new expenses.
- Remove existing expenses.
- Update existing expenses.

Let’s now add some constraints:

- You’ll be using **JWT** (JSON Web Token) to protect the endpoints and to identify the requester.
- For the different expense categories, you can use the following list (feel free to decide how to implement this as part of your data model):
  - Groceries

- Leisure
- Electronics
- Utilities
- Clothing
- Health
- Others.

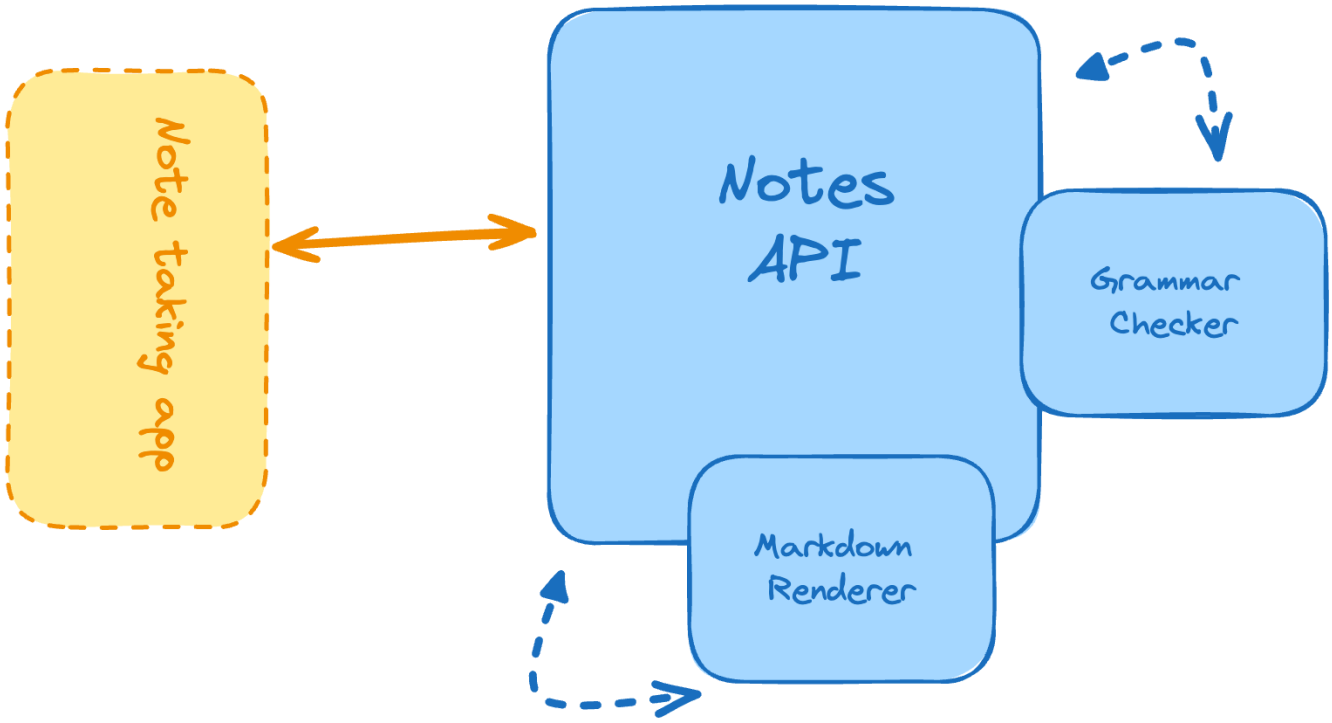
As a recommendation, you can use MongoDB or an ORM for this project, such as **Mongoose** (if you're using JavaScript/Node for this).

From everything you've done so far, you should feel pretty confident next time you have to build a new API.

5. Markdown Note-taking App

Difficulty: Moderate

Skills and technologies used: Text processing, Markdown libraries, persistent storage, REST API with file upload.



You've been building APIs all this time, so that concept alone should not be a problem by now. However, we're increasing the difficulty by allowing file uploads through your RESTful API. You'll need to understand how that part works inside a RESTful environment and then figure out a strategy to store those files while avoiding name collisions.

You'll also have to process the text in the following ways:

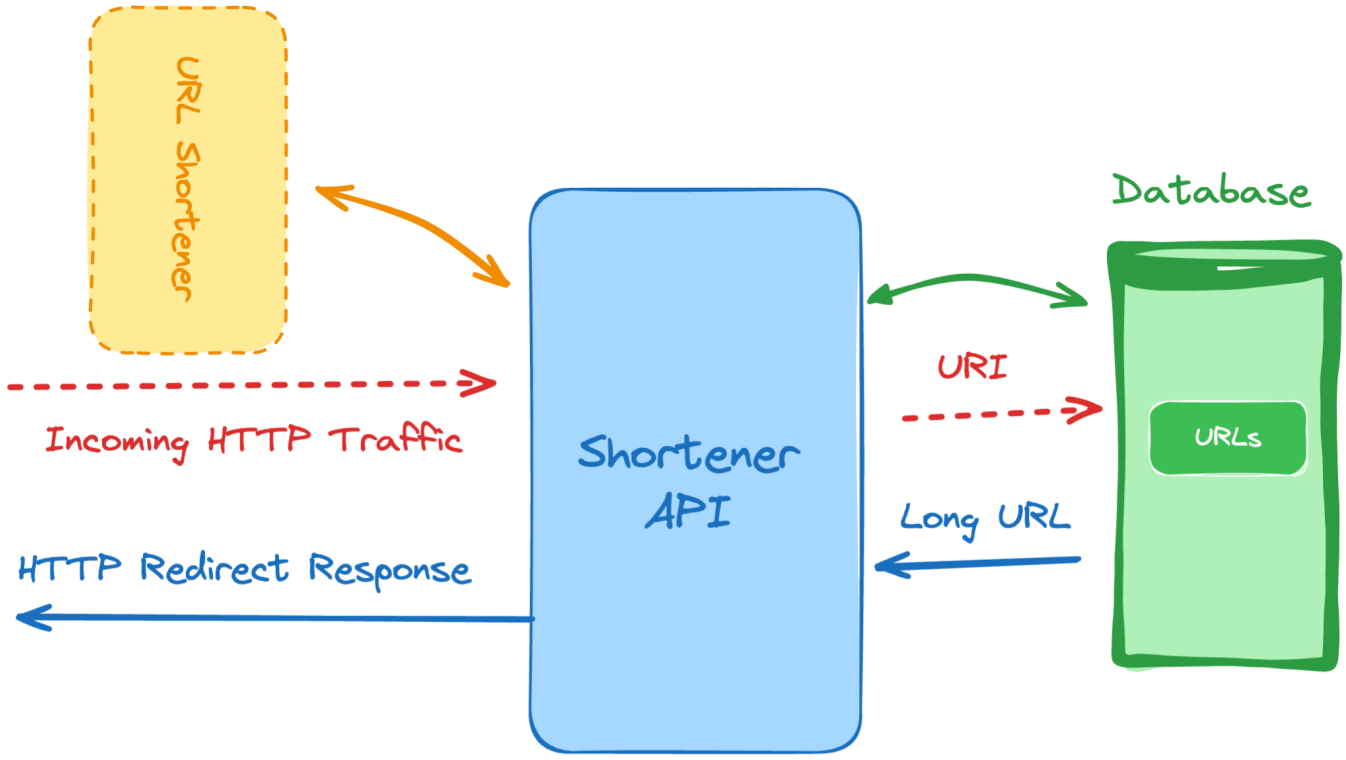
- You'll provide an endpoint to check the grammar of the note.
- You'll also provide an endpoint to save the note that can be passed in as Markdown text.
- Return the HTML version of the Markdown note (rendered note) through another endpoint.

As a recommendation, if you're using JavaScript for this particular project, you could use a library such as **Multer**, which is a Node.js module.

6. URL Shortening Service

Difficulty: Moderate

Skills and technologies used: Database indexing, HTTP redirects, RESTful endpoints



We're now moving away from your standard APIs, and tackling URL shortening. This is a very common service, which allows you to shorten very long URLs, especially when looking to share them on social media or make them easily memorable.

For this project idea let's focus on the following features, which you should be more than capable of implementing on your local environment, no matter your OS.

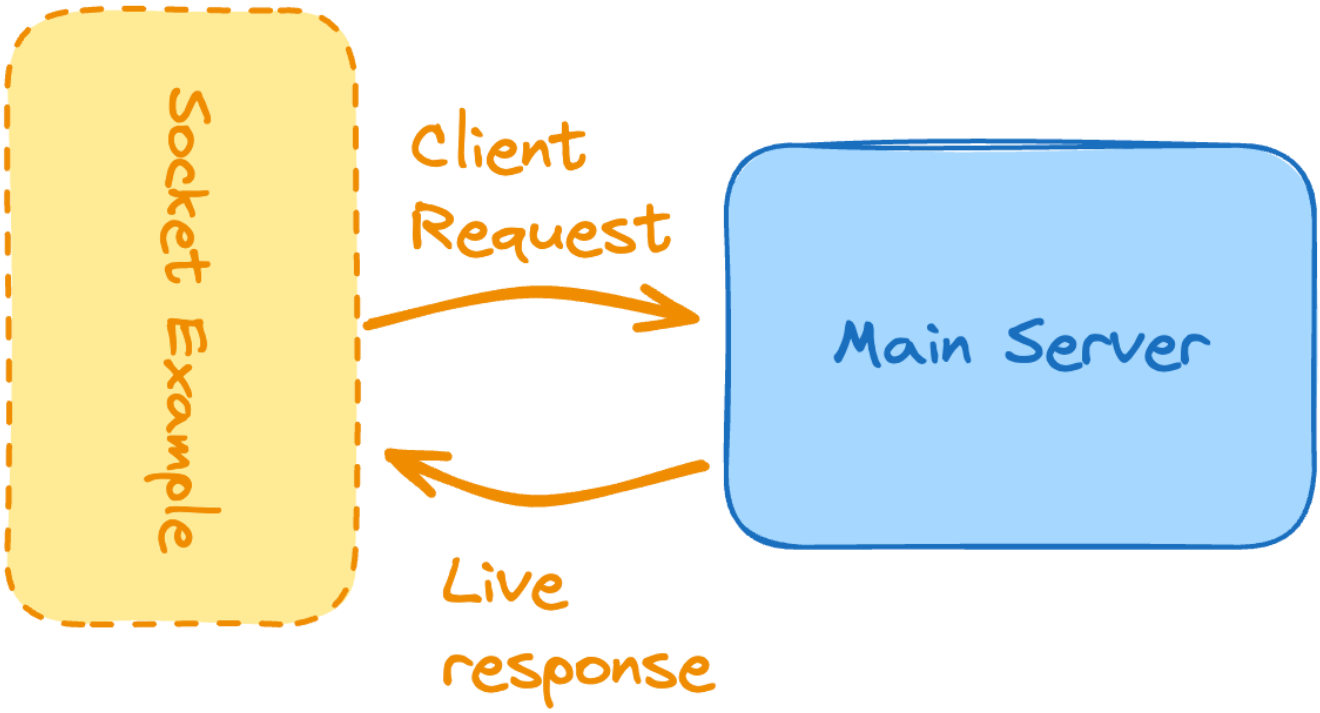
- Ability to pass a long URL as part of the request and get a shorter version of it. You're free to decide how you'll perform the shortening .

- Save the shorter and longer versions of the URL in the database to be used later during redirection.
- Configure a catch-all route on your service that gets all the traffic (no matter the URI used), finds the correct longer version and performs a redirection so the user is seamlessly redirected to the proper destination.

7. Real-time Polling App

Difficulty: Moderate

Skills and technologies used: WebSockets, live data updates, state management



Time to leave APIs alone for a while and focus on real-time interactions, another hot topic in web development. In fact, let's try to use some sockets.

Sockets are a fantastic way of enabling 2-way communication between two or more parties (systems) with very few lines of code. Read more about sockets **here**.

That being said, we're building both a client and a server for this project. The client can easily be a CLI (Command Line Interface) tool or a terminal program that will connect to the server and show the information being returned in real-time.

The flow for this first socket-based project is simple:

- The client connects to the server and sends a pre-defined request.

- The server upon receiving this request, will send, through the same channel, an automatic response.

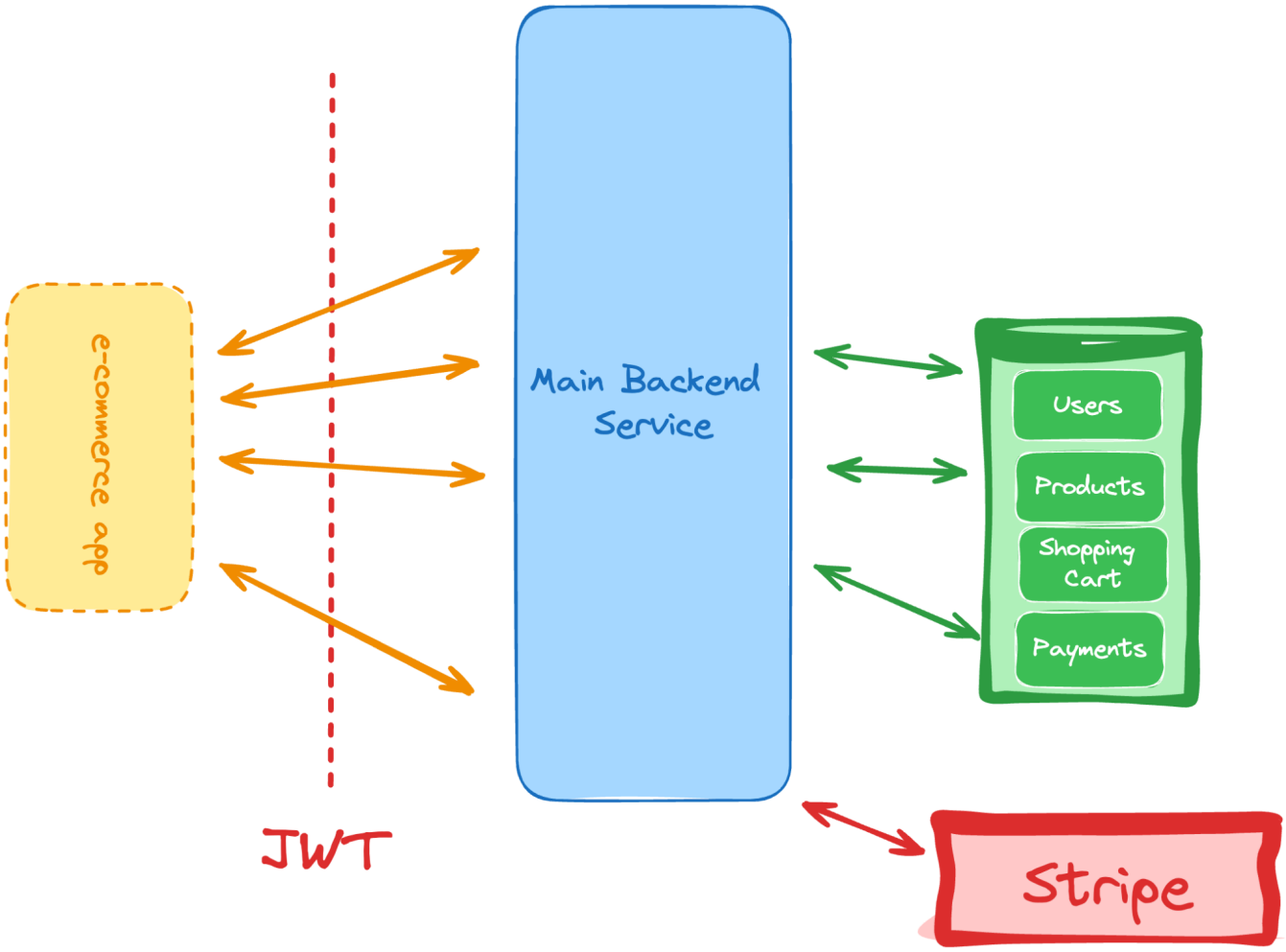
While the flow might seem very similar to how HTTP-based communication works, the implementation is going to be very different. Keep in mind that from the client perspective, the request is sent, and there is no waiting logic, instead, the client will have code that gets triggered when the message from the server is received.

This is a great first step towards building more complex socket-based systems.

8. Simple E-commerce API

Difficulty: Moderate

Skills and technologies used: Shopping cart logic, payment gateway integration (Stripe, PayPal), product inventory management



Back to the world of APIs, this time around we're pushing for a logic-heavy implementation.

For this one, you'll have to keep in mind everything we've been covering so far:

- JWT authentication to ensure many users can interact with it.
- Interaction with external services. Here you'll be integrating with payment gateways such as Stripe.
- A complex data model that can handle products, shopping carts, and more.

With that in mind, let's take a look at the responsibilities of this system:

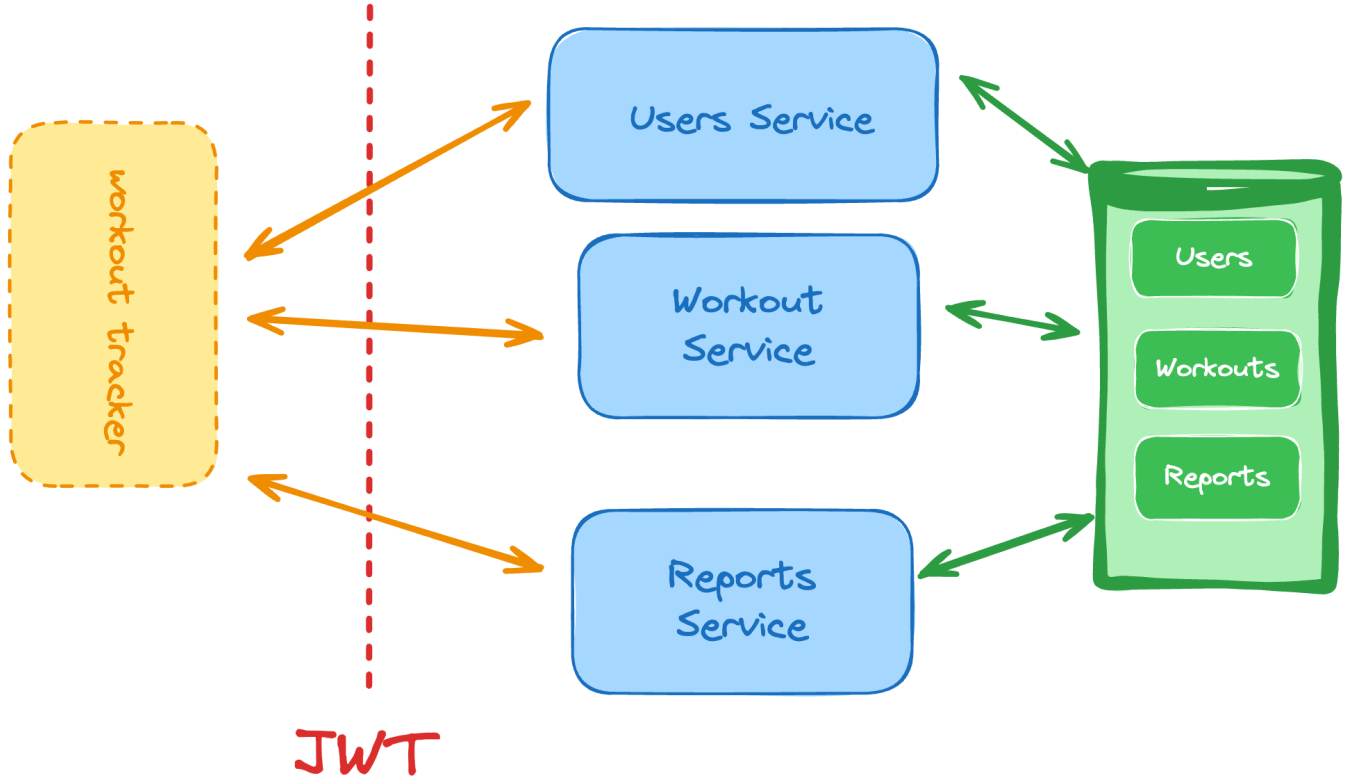
- JWT creation and validation to handle authorization.
- Ability to create new users.
- Shopping cart management, which involves payment gateway integration as well.
- Product listings.
- Ability to create and edit products in the database.

This project might not seem like it has a lot of features, but it compensates in complexity, so don't skip it, as it acts as a great progress check since it's re-using almost every skill you've picked up so far.

9. Fitness Workout Tracker

Difficulty: Moderate

Skills and technologies used: User-specific data storage, CRUD for workout sessions, date-time manipulation.



This backend project is not just about taking in user-generated notes, but rather, about letting users create their own workout schedules with their own exercises and then go through them, checking the ones they've done, and the ones they haven't.

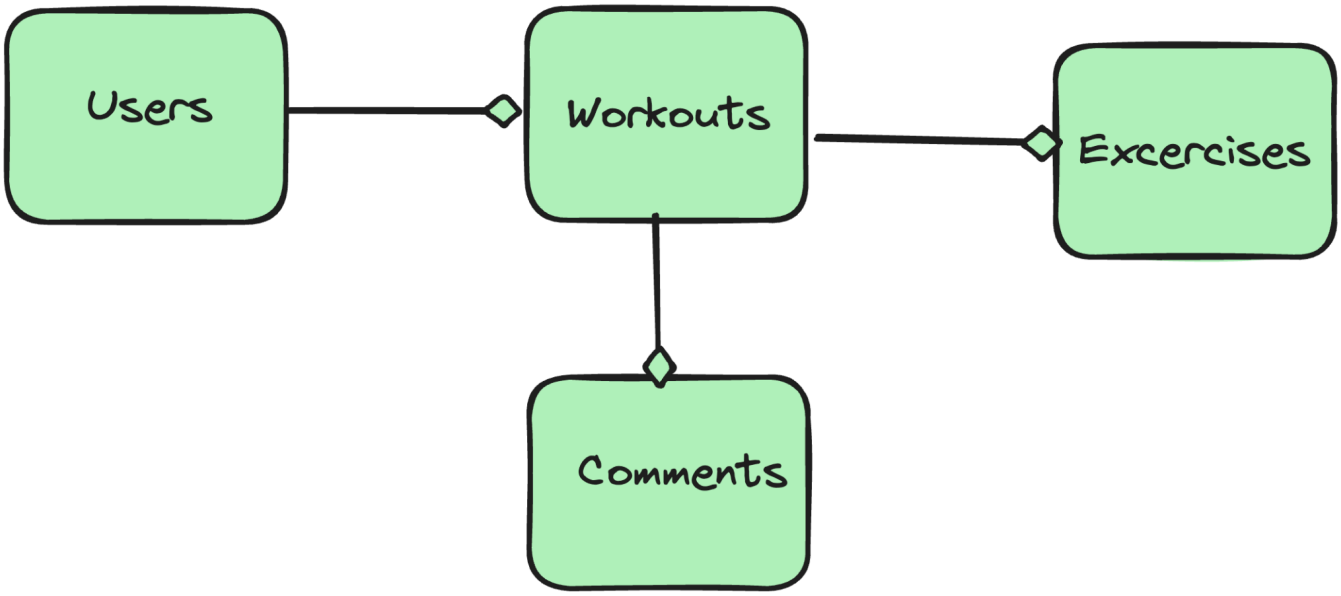
Making sure you also give them the space to add custom notes, with remarks about how the exercise in question felt and if they want to tweak it in the future.

Keep in mind the following responsibilities for this backend project:

- There needs to be a user sign-up and log-in flow in this backend system, as many users should be able to use it.
- There needs to be a secure JWT flow for authentication.
- The system should let users create workouts composed of multiple exercises.
- For each workout, the user will be able to update it and provide comments on it.
- The schedule the user creates needs to be associated to a specific date, and any listing of active or pending workouts needs to also be sorted by date (and time if you want to take it one step further).
- There should also be a report of past workouts, showing the percentage of finished workouts during the queried period.

The data model for this one can also be complex, as you'll have predefined exercises that need to be grouped into workout sessions, and those can then have associated comments (input from the

user).

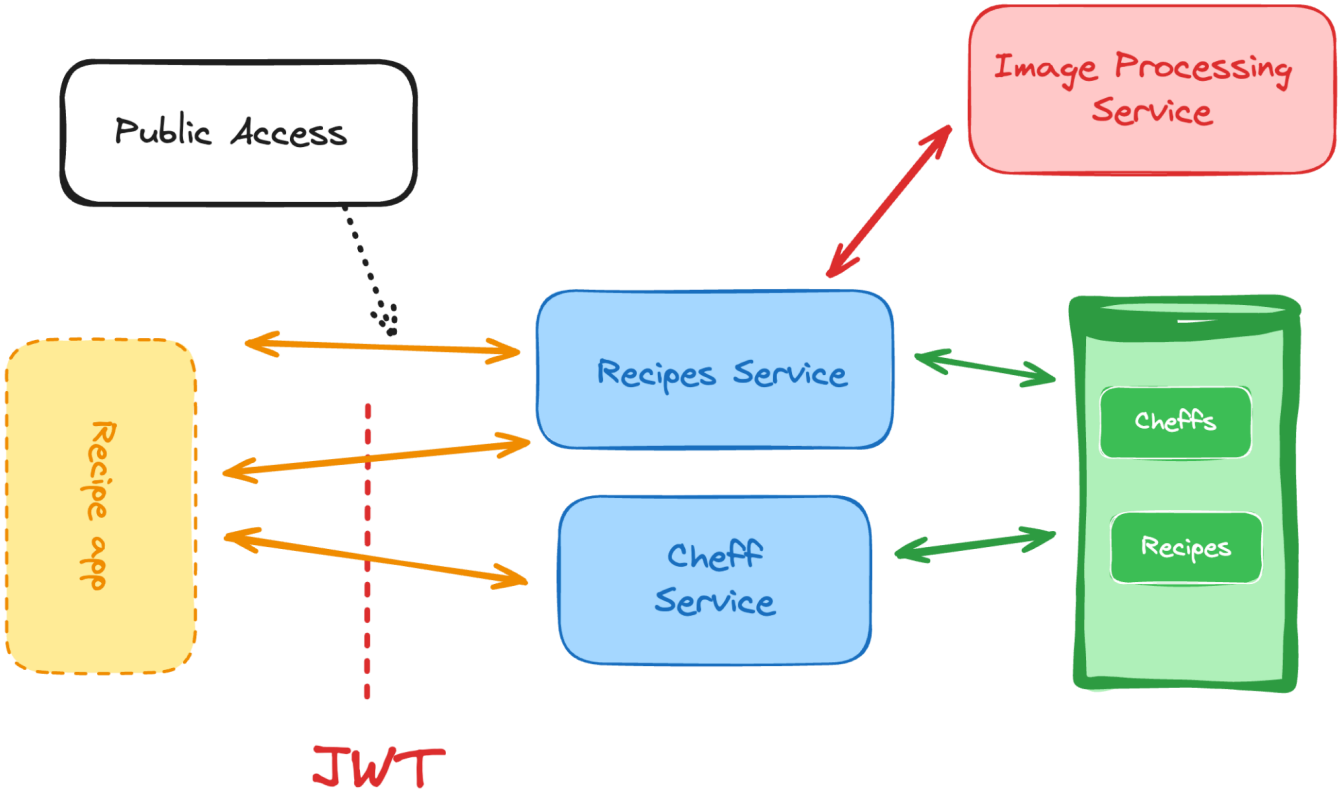


Consider the benefits of using a structured model here vs something document-based, such as MongoDB and pick the one that feels better for your implementation.

10. Recipe Sharing Platform

Difficulty: Moderate

Skills and technologies used: File uploads and image processing (like Sharp), user permissions, complex querying



While this project might feel a lot like the first one, the personal blogging platform, we're taking the same concept, and then adding a lot more on top of it.

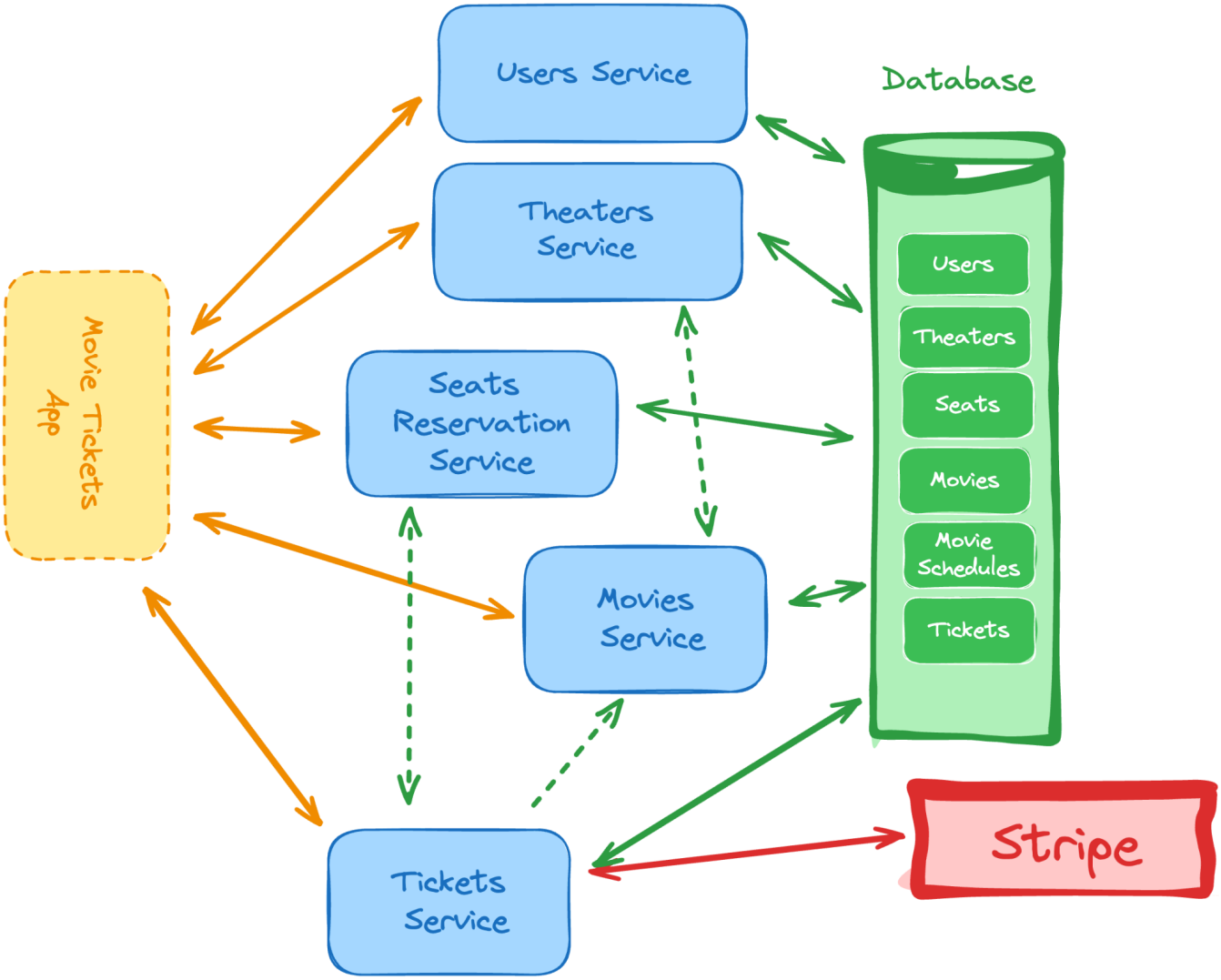
We're building a RESTful API (or rather several) that will let you perform the following actions:

- Access a list of recipes. You should be able to filter by keywords (text input), publication date, by chef, and by labels. Access to this endpoint should be public.
- The list should be paginated, and as part of the response on every page.
- Users should be able to sign up as chefs to the system to upload their own recipes.
- A JWT-secured login flow must be present to protect the endpoints in charge of creating new recipe posts.
- Images uploaded as part of the recipe should be processed to be re-sized into a standard size (you pick the dimensions). You can use a library such as **Sharp** for this.

11. Movie Reservation System

Difficulty: Difficult

Skills and technologies used: Relational data modeling (SQL), seat reservation logic, transaction management, schedule management.



There are very expensive pre-made tools that handle all this logic for companies, and the following diagram shows you a glimpse of that complexity.

As backend projects go, this one is a great example of the many different problems you might need to solve while working in web development.

A movie reservation system should allow any user to get tickets and their associated seats for any movie playing the specific day the user is looking to attend. This description alone already provides a lot of features and constraints we have to keep in mind:

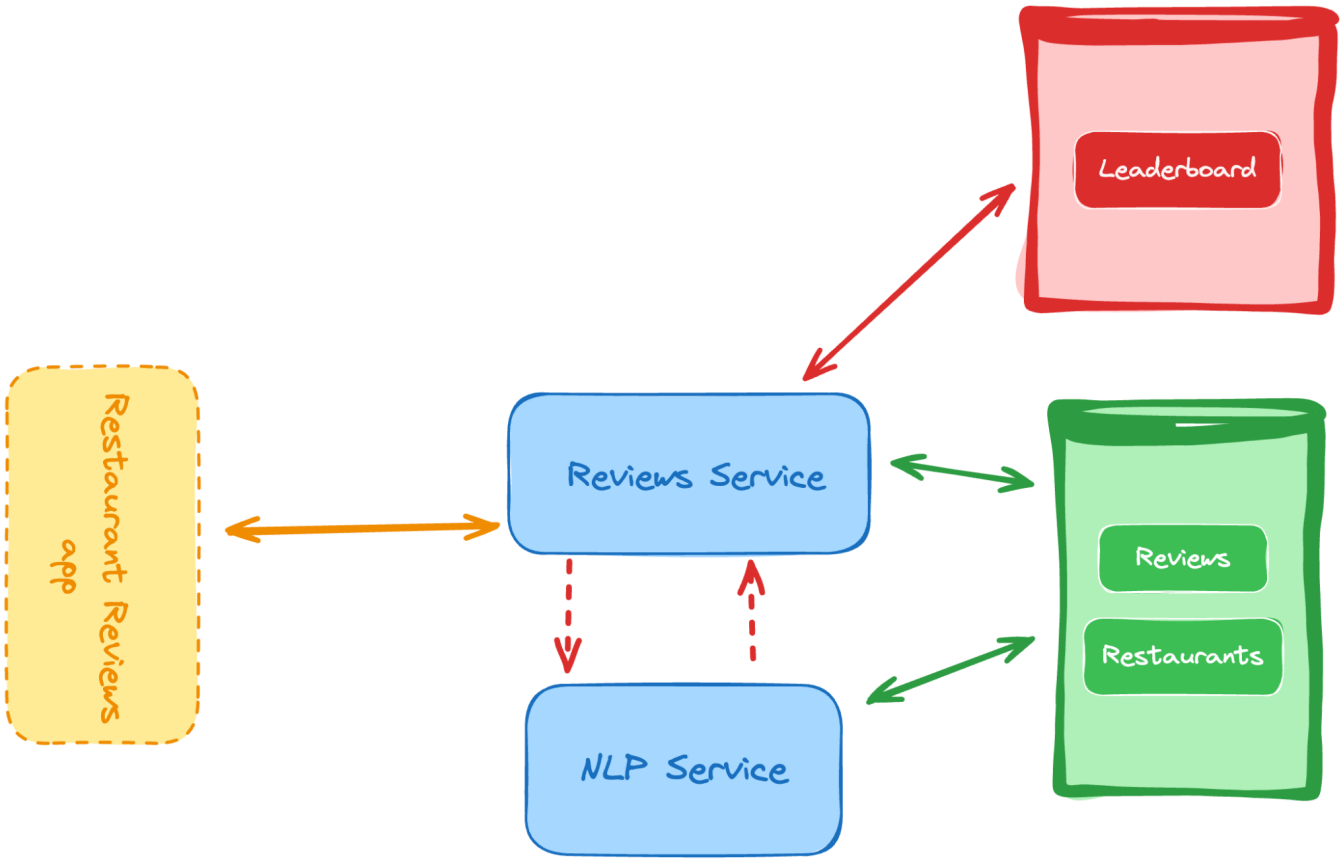
- We're going to have a list of movies (and maybe theaters as well).
- Each movie will have a recurring schedule for some time and then it'll be taken out to never return.
- Users should be able to list movies, search for them and filter by dates, genres and even actors.
- Once found, the user should be able to pick the seats for their particular movie of choice, and for their date of choice.

- This leads us to you having to keep a virtual representation of your movie theater to understand seating distribution and availability.
- In the end, the user should also be able to pay using an external payment gateway such as Stripe (we've already covered this step in the past).

12. Restaurant Review Platform (API) with automatic NLP analysis

Difficulty: Difficult

Skills and technologies used: RESTful API, In-memory database (for live leaderboard), SQL, Natural Language Processing to auto-label positive and negative comments.



Now this project takes a turn into the land of noSQL and AI by leading with user input. The aim of this particular backend project is to provide a very simple API that will let users:

- Input their own review of a restaurant (ideally, the API should request the restaurant's ID to make sure users are reviewing the correct one).
- Keep a leaderboard of restaurants with a generic positive or negative score, based on the type of reviews these restaurants get. For this, you can use Redis as an in-memory leaderboard to have your API query it, instead of hitting the database you're using. This also implies that you'll

have to keep the leaderboard updated on Redis as well (as a hint: look for type **SortedSet** data type to understand how to do this).

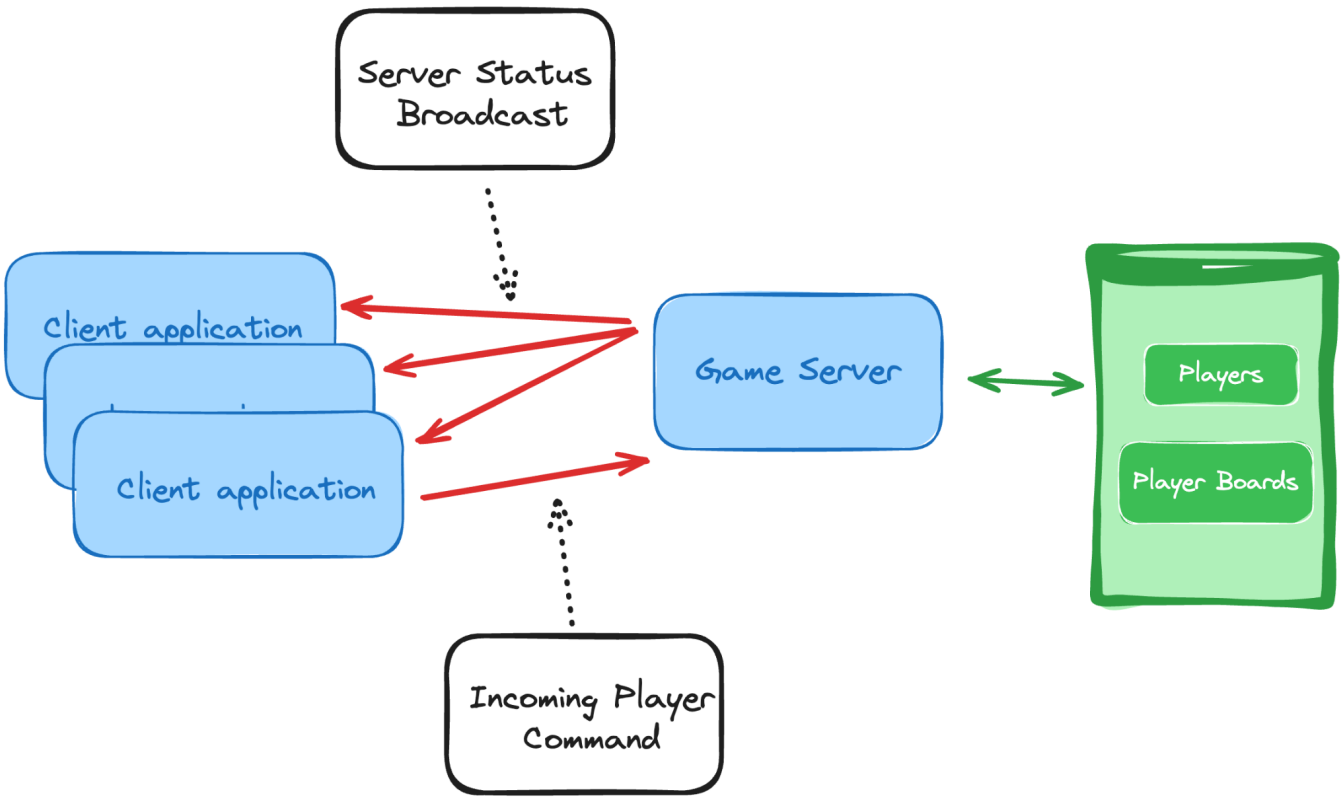
- Perform NLP (Natural Language Processing) on the user's text portion of the review, to understand if it's a positive one or a negative one.
- Use the result of the NLP as a scoring system for the leaderboard.

As a recommendation, you might want to use Python on this project, as there tend to be more libraries around NLP for this language.

13. Multiplayer Battleship Game Server

Difficulty: Difficult

Skills and technologies used: Game state synchronization, low-latency networking, concurrency control.



For this project you're not going to build a full game from scratch, so don't worry.

You will however, build a game server. Your game server will have to maintain the internal state of each player's board, and it should also enable communication between them by broadcasting their actions and results. Since we have "low-latency networking" as a constraint here, the logical implementation would be through the use of Sockets (so if you haven't done it yet, go back to project 7 and work on it first).

You're free to pick the programming language you feel more comfortable with, however, keep the mind that you'll have to:

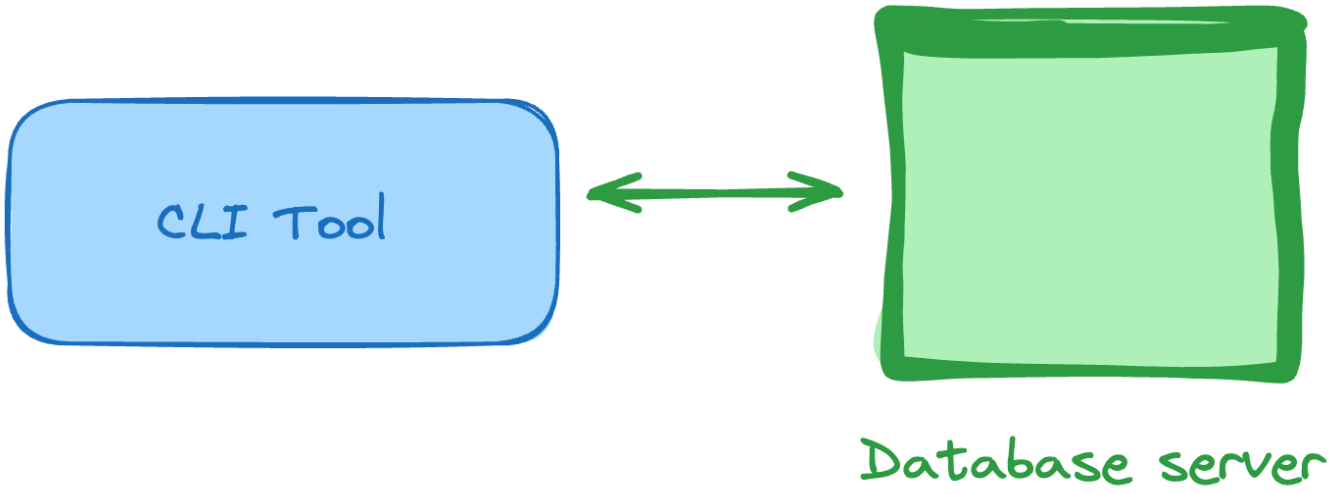
- Keep track of the player's state and game state.
- Enable 2-way communication between players and server.
- Allow players to join the game and set up their initial state somehow.

This can be a very fun project to work on, even if you're "just" building a terminal version of this multiplayer game, as you'll be using several of the concepts and technologies covered so far on this list.

14. Database Backup CLI utility

Difficulty: Difficult

Skills and technologies used: Advanced SQL, Database fundamentals, CLI development, Node.js (for CLI)



We're now moving away from the API world for a while, and into the world of command line interfaces, which is another very common space for backend developers to work on.

This time around, the project is a CLI utility to back up an entire database.

So for this project, you'll be creating a command line utility that takes the following attributes:

- **Host:** the host of your database (it can be localhost or anything else).
- **Username**: the utility will need a username to login and query the database.
- **Password**: same with the password, usually databases are protected this way.

- **DB Name**: the name of the database to backup. We're backing up the entire set of tables inside this database.
- **Destination folder**: the folder where all the dump files will be stored.

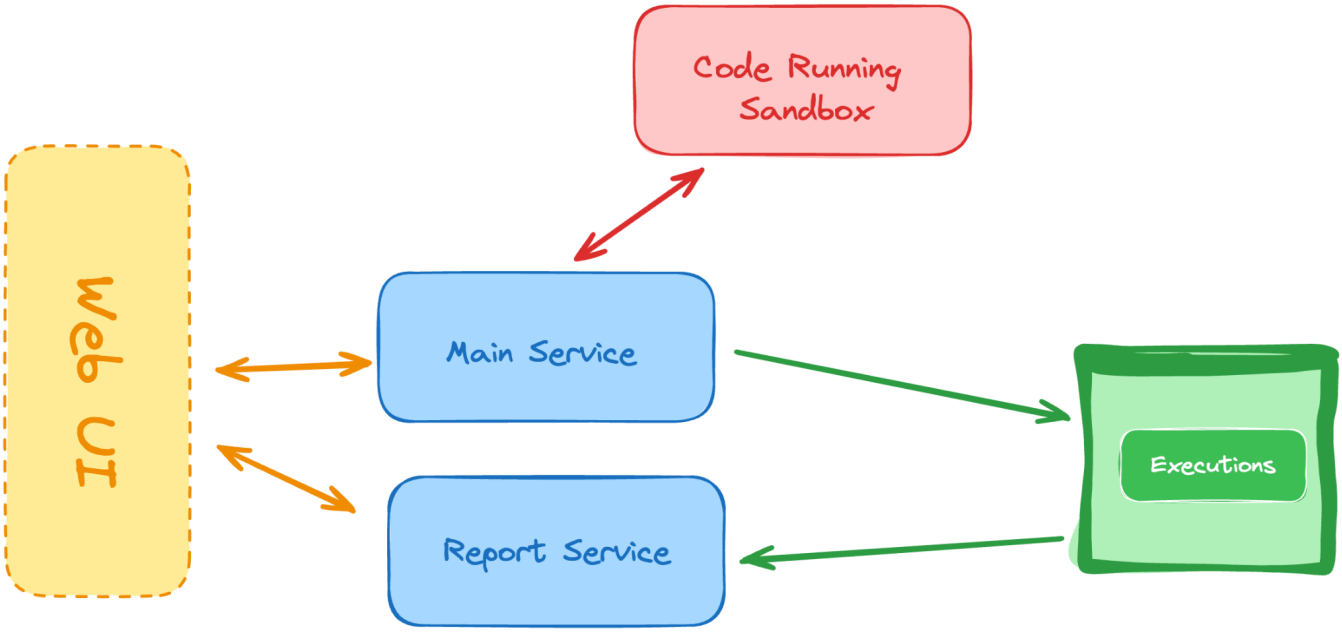
With all that information, your utility should be able to connect to the database, pull the list of tables, and for each one understand its structure and its data. In the end, the resulting files inside the destination folder should have everything you need to restore the database on another server simply by using these files.

Finally, if you haven't heard of it yet, you might want to check out the **SHOW CREATE TABLE** statement.

15. Online Code Compiler API

Difficulty: Difficult

Skills and technologies used: Sandboxing code execution, integration with compilers, WebSocket communication.



For this project, you'll be building the backend of a remote code execution application. In other words, your APIs will allow you to receive source code written using a specific language of choice (you can pick the one you want, and only allow that one), run it and then return the output of that execution.

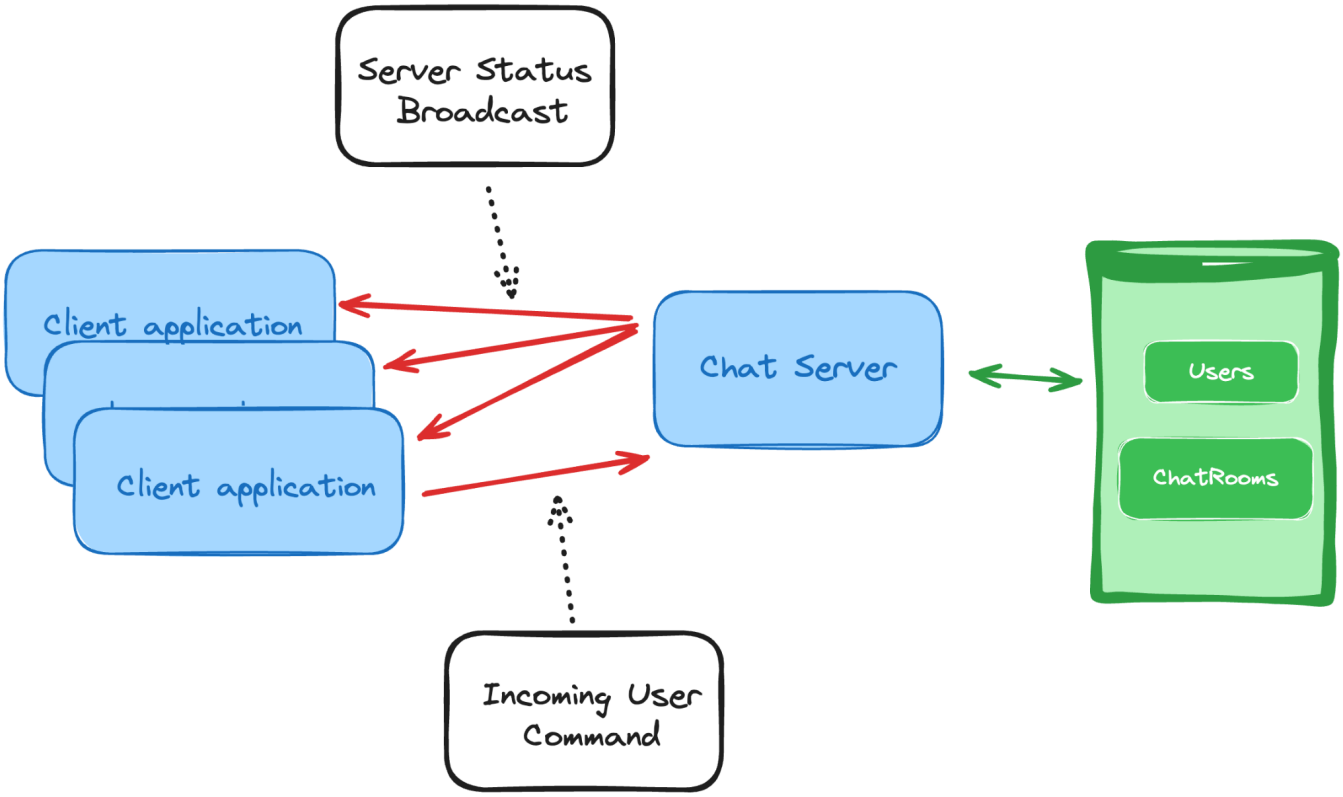
Of course, doing this without any restrictions is not worth it for being in the "difficult" section of this list, so let's kick it up a notch:

- The code execution should be done inside a safe sandbox, which means that the code can't hurt or affect the system it's running on, no matter what the code or the logic dictates.
- On top of that, for long-running tasks, the API should also provide a status report containing the following information:
- Time running.
- Start time of the execution.
- Lines of code being executed.

16. Messaging Platform Backend

Difficulty: Difficult

Skills and technologies used: Real-time messaging, end-to-end encryption, contact synchronization



Yes, we're talking about a chat platform here. And as a backend developer you're more than ready to implement both the server and the client application.

This backend project would take project #7 to the next level, by implementing the following responsibilities:

- Adding message encryption between client applications
- The ability to understand who's online

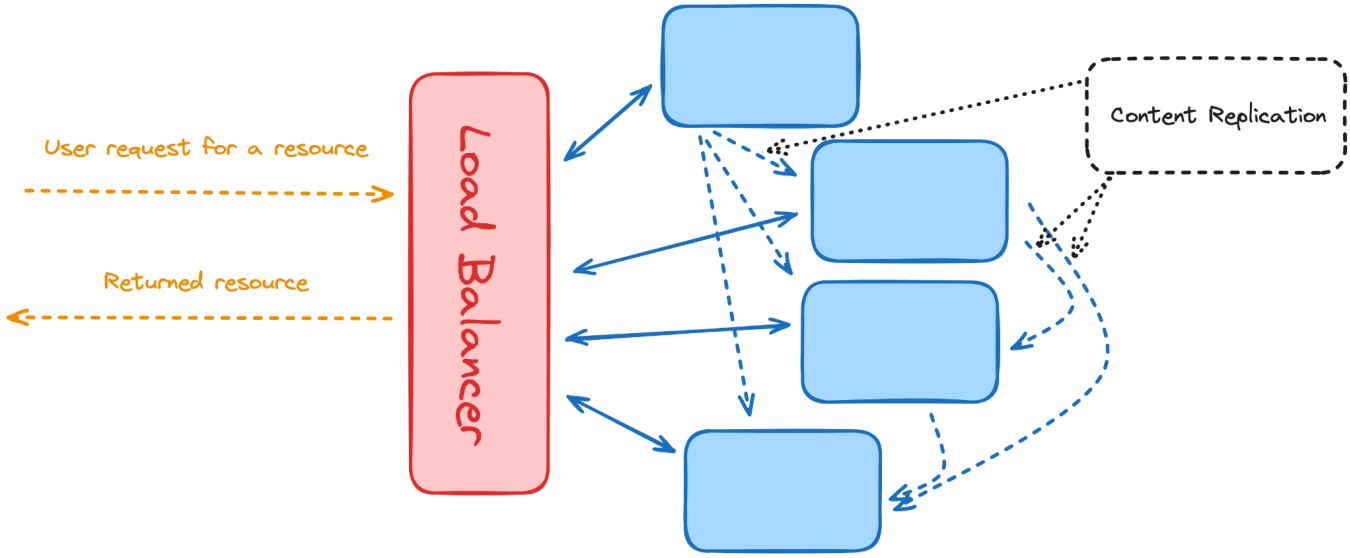
- Understand if those users are interacting with you (a.k.a showing the "[username] is typing" message in real-time).
- Sending a message from one of the clients into the server should be broadcasted to the rest of the clients connected.

As a recommendation for technology implementing this project, **Socket.io** would be a perfect match. This means you'd be using JavaScript (node.js) for this.

17. Content Delivery Network (CDN) Simulator

Difficulty: Very Difficult

Skills and technologies used: Load balancing algorithms, caching strategies, network latency simulation



For this particular backend project, we're not going to focus on coding, but rather on backend tools and their configuration. A **CDN** (or Content Delivery Network) is a platform that allows you to serve static content (like text files, images, audio, etc) safely and reliably.

Instead of having all files inside the same server, the content is replicated and distributed across a network of servers that can provide you with the files at any given point in time.

The point of this project is for you to figure out a way to set up your own CDN keeping in mind the following points:

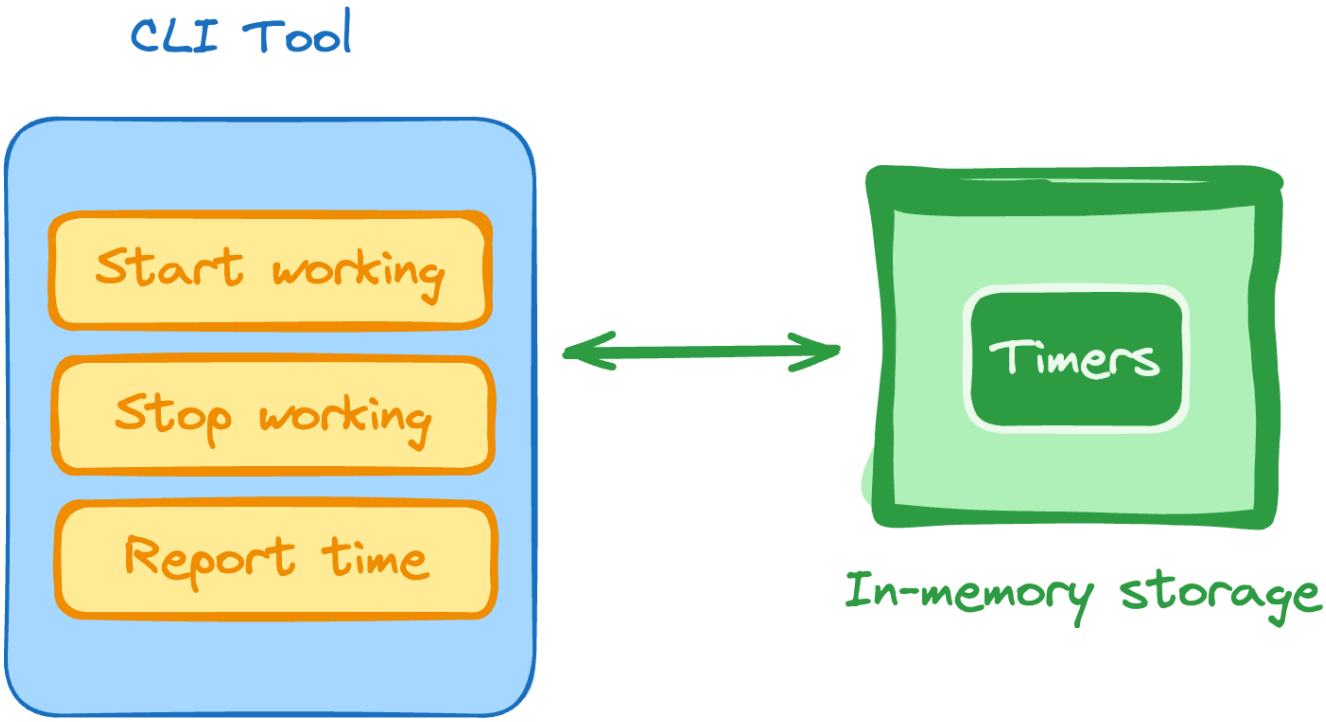
- Use cloud servers (you can pick your favorite cloud provider for this)
- Configure a load balancer to distribute the load between all servers.
- Set up a caching strategy.

Remember that all major cloud providers have a free tier that allows you to use all their services for some time. AWS for example, allows for a full year of free tier limited to the type of resources you can use.

18. Time-tracking CLI for Freelancers

Difficulty: Very Difficult

Skills and technologies used: time tracking, interactive CLI, Day.js for time operations, reporting.



As freelancers, sometimes understanding what you're working on, or understanding how much time you've spent on a particular project once it's time to get paid, can be a challenge.

So, with this command line interface tool, we'll try to solve that pain for freelancers. The tool you're developing should let you specify that you're starting to work on a project, and once you're done, you should also be able to say that you've stopped.

On top of that, there should be an interactive reporting mode that should tell you the amount of time spent so far on a particular project (with the ability to filter by date and time), so you can know exactly how much to charge each client.

From the user's POV, you could have commands like this:

- freelance start project1
- freelance stop project2

And when in interactive mode, something like this should work:

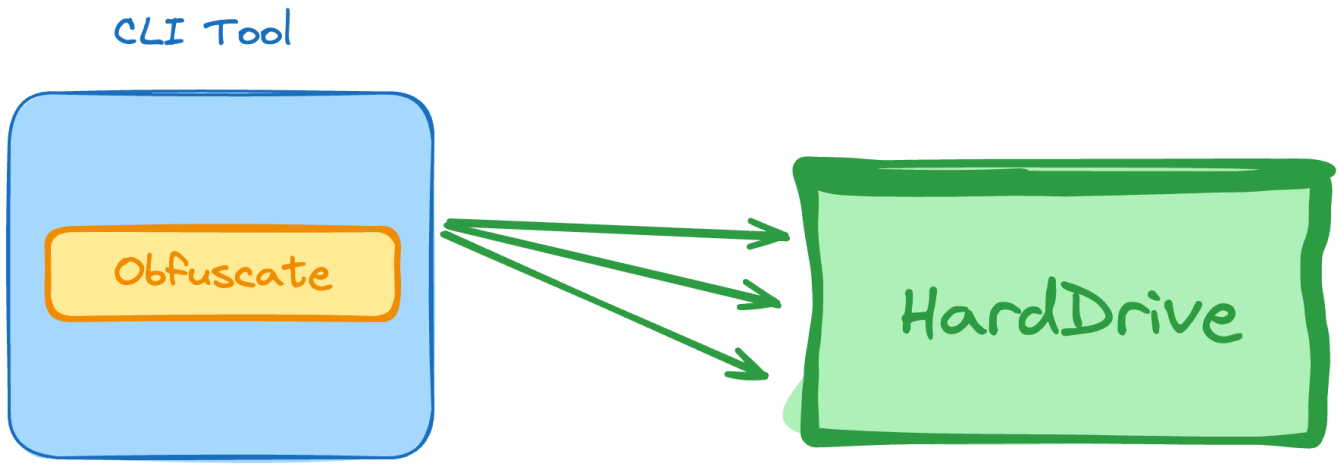
- report project1 since 1/2/24

The challenge on this backend project is not just the CLI itself, which you've built in the past, but the actual time tracking logic that needs to happen internally. You'll be keeping track of small chunks of time associated with different backend projects, and once the report is requested, you need to properly query your DB and get only the right chunks, so you can later add them up and return a valid number.

19. JS Obfuscator CLI utility

Difficulty: Very Difficult

Skills and technologies used: code obfuscation, batch processing of files using a CLI, Node.js.



Code obfuscation happens when you turn a perfectly readable code into something that only a machine can understand, without changing the plain text nature of the file. In other words, you just make it impossible for a human to read and understand.

Many tools do this in the JS ecosystem, it's now your turn to create a new tool and perform the exact same action. As an added difficulty, you'll be coding a tool that does this to an entire folder filled with files (not just one at the time).

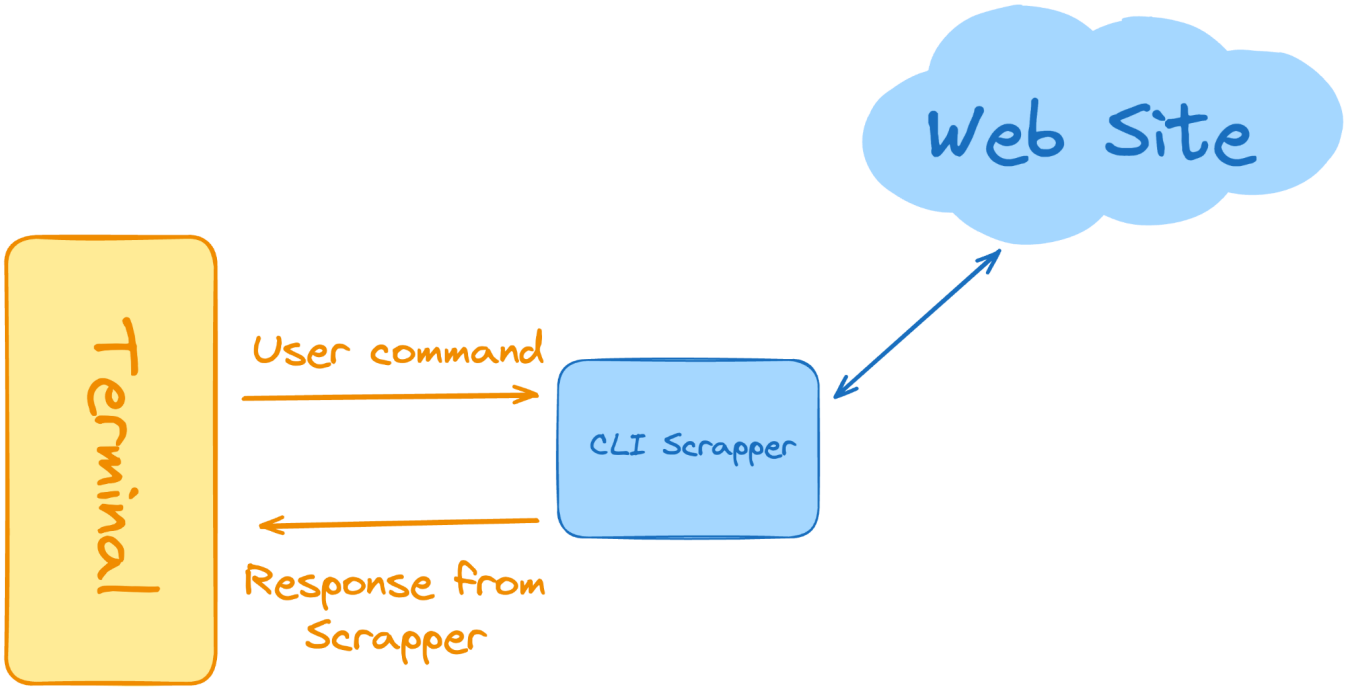
Make sure the output for each file is placed inside the same folder, with a ".obs.js" extension, and that you're also navigating sub-folders searching for more files.

Try to avoid libraries that already perform these exact same tasks, as you'll be skipping through all the potential problems you can find, and effectively learning nothing from the experience.

20. Web Scraper CLI

Difficulty: Very Difficult

Skills and technologies used: Web scraping, headless browsing, rules engine



A web scraper is a tool that allows you to navigate a website through code, and in the process, capture information from the presented web pages.

As part of the last backend project of this list, you'll be implementing your very own web scraper CLI tool. This tool will take input from the user with a list of preset commands, such as:

- show code: to list the HTML code of the current page.
- navigate: to open a new URL
- capture: this will return a subsection of the HTML of the current page using the CSS selector you specify.
- click on: this command will trigger a click on a particular HTML element using a CSS selector provided.

Feel free to add extra commands to make the navigation even more interactive.

With the last of our backend project ideas, you've covered all the major areas involved in backend development and you're more than ready to apply for a backend development job if you haven't already.

If you find a piece of technology that wasn't covered here, you'll have the skills required to pick it up in no time.

Join the Community

roadmap.sh is the 7th most starred project on GitHub and is visited by hundreds of thousands of developers every month.

Rank 7th out of 28M!

301K

GitHub Stars

Star us on GitHub
Help us reach #1

+90k every month

+1.5M

Registered Users

Register yourself
Commit to your growth

+2k every month

30K

Discord Members

Join on Discord
Join the community

Roadmaps Best Practices Guides Videos FAQs YouTube

roadmap.sh by @kamrify

Community created roadmaps, best practices, projects, articles, resources and journeys to help you choose your path and grow in your career.

THENEWSTACK

The top DevOps resource for Kubernetes, cloud-native computing, and large-scale development and deployment.

DevOps · Kubernetes · Cloud-Native